

OPT11: Optimization of a Data-Driven Algorithmic Trading Strategy Using Hyperparameter Search

Arshia Barootkoob Dezfooli

Final-Semester Bachelor of Software Engineering, Seneca Polytechnic, Canada

Abstract

Algorithmic trading systems frequently rely on manually tuned technical indicators, which may limit adaptability across assets and time horizons. This work presents OPT11, a data-driven trading framework that applies automated hyperparameter optimization to identify effective combinations of technical indicators. Historical financial data were obtained using the Yahoo Finance API and processed to generate trading signals based on Simple Moving Averages (SMA) and the Relative Strength Index (RSI). The Optuna optimization framework was employed to search parameter spaces efficiently, optimizing strategy performance on training data and validating results via out-of-sample backtesting. The system was implemented in Python and augmented with an interactive Gradio-based interface to support experimentation. Results demonstrate that systematic optimization can significantly improve strategy robustness and performance compared to fixed, manually selected parameters.

1. Introduction

Financial markets generate large volumes of time-series data, making them suitable candidates for data-driven and algorithmic approaches. Traditional rule-based trading strategies often rely on fixed technical indicators whose parameters are chosen heuristically or based on prior experience. Such approaches may fail to generalize across assets or adapt to changing market conditions.

Recent advances in machine learning and optimization frameworks enable the automated exploration of large parameter spaces. This project investigates whether hyperparameter optimization techniques can be effectively applied to technical-indicator-based trading

strategies. The primary objective of OPT11 is to construct an end-to-end system that integrates data acquisition, strategy definition, automated optimization, and evaluation within a reproducible framework.

2. Related Work

Technical analysis indicators such as moving averages and momentum oscillators remain widely used in algorithmic trading systems. Simple Moving Average (SMA) crossover strategies capture trend direction, while the Relative Strength Index (RSI) provides insight into momentum and overbought conditions. While these indicators are well studied, their effectiveness is highly dependent on parameter selection.

Hyperparameter optimization methods, including grid search, Bayesian optimization, and evolutionary strategies, have been applied in financial modeling to improve predictive models and trading rules. Optuna is a modern optimization framework that implements efficient sampling strategies and pruning to accelerate the search process. OPT11 builds on these ideas by applying automated hyperparameter search to a classical indicator-based trading framework.

3. Data and Preprocessing

Historical price data were collected programmatically using the Yahoo Finance API. The dataset consists of time-indexed closing prices for user-specified financial instruments. Prior to analysis, timestamps were normalized to a consistent time zone and values were checked for missing or invalid entries.

The dataset was divided into training and testing subsets using a fixed split ratio of 80% for optimization and 20% for evaluation. This separation ensures that optimization results are evaluated on unseen data, reducing the risk of overfitting.

4. Trading Strategy Design

4.1 Indicators

The trading strategy integrates two widely used technical indicators:

- **Simple Moving Average (SMA):** Used to identify trend direction by comparing short-term and long-term price averages.
- **Relative Strength Index (RSI):** A momentum oscillator used to identify overbought conditions.

Buy signals are generated when the short-term SMA exceeds the long-term SMA while the RSI remains below a configurable overbought threshold.

4.2 Signal Execution

Signals are translated into discrete trades using a rule-based execution mechanism. Positions are opened at the next available time step after a signal and closed when the signal condition is no longer satisfied. Trade performance is measured in terms of profit and loss (P&L), number of trades, and minimum equity required.

5. Optimization Methodology

Hyperparameter optimization was performed using the Optuna framework. The following parameters were optimized:

- Short SMA window length
- Long SMA window length
- RSI calculation period
- RSI overbought threshold

Optuna employs a trial-based optimization process, proposing parameter configurations and evaluating them using a defined objective function. The objective function in OPT11 maximizes cumulative P&L on the training dataset. A median-based pruner was used to terminate unpromising trials early, improving computational efficiency.

6. Experimental Results

Experiments were conducted across multiple assets and temporal ranges (3-month, 6-month, and 1-year windows). Optimized parameter sets differed across assets and time horizons, indicating that no single configuration is universally optimal.

Out-of-sample results demonstrated improved performance compared to non-optimized baseline strategies. The system consistently produced positive cumulative P&L under favorable market conditions, while the calculation of minimum equity requirements provided insight into risk exposure.

7. System Implementation

The system was implemented entirely in Python using Pandas and NumPy for data processing, Optuna for optimization, and Gradio for user interaction. The graphical interface allows users to input multiple tickers and generate optimization reports interactively, enhancing usability and reproducibility.

8. Discussion

The results suggest that automated hyperparameter optimization can substantially enhance the effectiveness of traditional technical-indicator-based trading strategies. While OPT11 does not employ predictive machine learning models, the optimization process itself reflects machine-learning-oriented experimentation and validation practices.

Limitations include sensitivity to market regimes and transaction costs, which were not explicitly modeled. Future extensions could incorporate risk-adjusted objectives, predictive models, or reinforcement learning techniques.

9. Conclusion

OPT11 demonstrates the viability of data-driven optimization frameworks for algorithmic trading. By combining classical technical indicators with modern optimization techniques, the system provides an adaptable and reproducible approach to strategy development. This work highlights the intersection of data analysis, machine-learning-inspired optimization, and financial modeling, and serves as a foundation for further exploration in intelligent decision-making systems.

References

1. Murphy, J. J. *Technical Analysis of the Financial Markets*. New York Institute of Finance.
2. Akiba, T. et al. "Optuna: A Next-generation Hyperparameter Optimization Framework."
3. Yahoo Finance API Documentation.

```

!pip install optuna
import yfinance as yf
import pandas as pd
import numpy as np
import optuna
import io
from contextlib import redirect_stdout
!pip install gradio --quiet
import gradio as gr
from datetime import datetime, timedelta

Requirement already satisfied: optuna in
/usr/local/lib/python3.12/dist-packages (4.8.0)
Requirement already satisfied: alembic>=1.5.0 in
/usr/local/lib/python3.12/dist-packages (from optuna) (1.18.4)
Requirement already satisfied: colorlog in
/usr/local/lib/python3.12/dist-packages (from optuna) (6.10.1)
Requirement already satisfied: numpy in
/usr/local/lib/python3.12/dist-packages (from optuna) (2.0.2)
Requirement already satisfied: packaging>=20.0 in
/usr/local/lib/python3.12/dist-packages (from optuna) (26.1)
Requirement already satisfied: sqlalchemy>=1.4.2 in
/usr/local/lib/python3.12/dist-packages (from optuna) (2.0.49)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-
packages (from optuna) (4.67.3)
Requirement already satisfied: PyYAML in
/usr/local/lib/python3.12/dist-packages (from optuna) (6.0.3)
Requirement already satisfied: Mako in /usr/local/lib/python3.12/dist-
packages (from alembic>=1.5.0->optuna) (1.3.11)
Requirement already satisfied: typing-extensions>=4.12 in
/usr/local/lib/python3.12/dist-packages (from alembic>=1.5.0->optuna)
(4.15.0)
Requirement already satisfied: greenlet>=1 in
/usr/local/lib/python3.12/dist-packages (from sqlalchemy>=1.4.2-
>optuna) (3.4.0)
Requirement already satisfied: MarkupSafe>=0.9.2 in
/usr/local/lib/python3.12/dist-packages (from Mako->alembic>=1.5.0-
>optuna) (3.0.3)

def compute_rsi(series, period):
    delta = series.diff()
    gain = (delta.where(delta > 0, 0)).rolling(window=period).mean()
    loss = (-delta.where(delta < 0, 0)).rolling(window=period).mean()
    rs = gain / loss
    rsi = 100 - (100 / (1 + rs))
    return rsi

def backtest_strategy(data, short_ma, long_ma, rsi_period,
rsi_overbought, period, position_size=1):

```

```

data['short_ma'] = data['Close'].rolling(short_ma).mean()
data['long_ma'] = data['Close'].rolling(long_ma).mean()
data['rsi'] = compute_rsi(data['Close'], rsi_period)
data['signal'] = (data['short_ma'] > data['long_ma']) &
(data['rsi'] < rsi_overbought)
data['position'] =
data['signal'].shift(1).astype(bool).fillna(False)

trades = []
in_trade = False
entry_price = None
entry_date = None
open_position = None

for i in range(1, len(data)):
    if data['position'].iloc[i] and not in_trade:
        entry_price = data['Close'].iloc[i]
        entry_date = data.index[i] if isinstance(data.index,
pd.DatetimeIndex) else data['Date'].iloc[i]
        in_trade = True
    elif (not data['position'].iloc[i] and in_trade):
        #elif (not data['position'].iloc[i] and in_trade) or (in_trade
and data['Close'].iloc[i]>=entry_price*1.02) or (in_trade and
data['Close'].iloc[i]<=entry_price*0.99):
        exit_price = data['Close'].iloc[i]
        exit_date = data.index[i] if isinstance(data.index,
pd.DatetimeIndex) else data['Date'].iloc[i]
        trades.append({
            'entry_date': entry_date.strftime('%Y-%m-%d %H:%M'),
            'exit_date': exit_date.strftime('%Y-%m-%d %H:%M'),
            'entry_price': float(entry_price),
            'exit_price': float(exit_price),
            'pnl': round((float(exit_price) - float(entry_price))
* position_size, 2)
        })
        in_trade = False

# Check for open position at the end
if in_trade:
    open_position = {
        'entry_date': entry_date.strftime('%Y-%m-%d %H:%M'),
        'exit_date': None,
        'entry_price': float(entry_price),
        'exit_price': None,
        'pnl': None
    }
    trades.append(open_position)
    print("\nOpen position:")
    print(open_position)
else:

```

```

        print("\nNo open position.")

    print(f"\nNumber of Trades: {len(trades)}")
    print("Trade List:")
    for trade in trades:
        print(trade)

    # Calculate minimum equity ever needed
    running_balance = 0.0
    min_equity_needed = 0.0

    for trade in trades:
        if trade['exit_price'] is not None: # Only closed trades
            affect running balance
            running_balance -= trade['entry_price'] * position_size
            if running_balance < min_equity_needed:
                min_equity_needed = running_balance
            running_balance += trade['exit_price'] * position_size

    min_equity_needed = abs(min_equity_needed) if min_equity_needed <
0 else 0.0001
    print(f"\nMinimum Equity Ever Needed: {min_equity_needed:.2f} unit
of currency")

    # Calculate total P&L (only for closed trades)
    total_pnl = sum(trade['pnl'] for trade in trades if trade['pnl']
is not None)
    print(f"\nTotal P&L on min equity: {total_pnl:.2f} unit of
currency")
    if period == '3mo':
        a=18
    elif period == '6mo':
        a=36
    elif period == '1y':
        a=73
    print(f"Total End Change on any equity:
{round((total_pnl+min_equity_needed)/min_equity_needed,2)} percent in
the last {a} days.")
    return round(total_pnl,2)

def backtest(data, short_ma, long_ma, rsi_period, rsi_overbought,
position_size=1):
    data['short_ma'] = data['Close'].rolling(short_ma).mean()
    data['long_ma'] = data['Close'].rolling(long_ma).mean()
    data['rsi'] = compute_rsi(data['Close'], rsi_period)
    data['signal'] = (data['short_ma'] > data['long_ma']) &
(data['rsi'] < rsi_overbought)
    data['position'] =

```



```

data['signal'].shift(1).astype(bool).fillna(False)

trades = []
in_trade = False
entry_price = None
entry_date = None
open_position = None

for i in range(1, len(data)):
    if data['position'].iloc[i] and not in_trade:
        entry_price = data['Close'].iloc[i]
        entry_date = data.index[i] if isinstance(data.index,
pd.DatetimeIndex) else data['Date'].iloc[i]
        in_trade = True
    elif (not data['position'].iloc[i] and in_trade):
        #elif (not data['position'].iloc[i] and in_trade) or (in_trade
and data['Close'].iloc[i]>=entry_price*1.02) or (in_trade and
data['Close'].iloc[i]<=entry_price*0.99):
        exit_price = data['Close'].iloc[i]
        exit_date = data.index[i] if isinstance(data.index,
pd.DatetimeIndex) else data['Date'].iloc[i]
        trades.append({
            'entry_date': entry_date,
            'exit_date': exit_date,
            'entry_price': entry_price,
            'exit_price': exit_price,
            'pnl': (exit_price - entry_price) * position_size
        })
        in_trade = False

# Check for open position at the end
if in_trade:
    open_position = {
        'entry_date': entry_date,
        'exit_date': None,
        'entry_price': entry_price,
        'exit_price': None,
        'pnl': None
    }
    trades.append(open_position)

# Calculate minimum equity ever needed
running_balance = 0.0
min_equity_needed = 0.0

for trade in trades:
    if trade['exit_price'] is not None: # Only closed trades
affect running balance
        running_balance -= trade['entry_price'] * position_size
        if running_balance < min_equity_needed:

```

```

        min_equity_needed = running_balance
        running_balance += trade['exit_price'] * position_size

    # Calculate total P&L (only for closed trades)
    total_pnl = sum(trade['pnl'] for trade in trades if trade['pnl']
is not None)
    return round(total_pnl,2)

def objective(trial, data):
    short_ma = trial.suggest_int("short_ma", 3, 50) #1 100
    long_ma = trial.suggest_int("long_ma", 50, 300) #1 500
    rsi_period = trial.suggest_int("rsi_period", 10, 30) #1 200
    rsi_overbought = trial.suggest_int("rsi_overbought", 60, 90) #1
100
    return backtest(data.copy(), short_ma, long_ma, rsi_period,
rsi_overbought)

def operation(period, trial, interval='1h'):
    period = period

    data = yf.download(ticker, period=period, interval=interval,
prepost=True)

    data.index = data.index.tz_convert('America/Toronto')
    data = data['Close'].round(4)

    data = data.reset_index() # Move the datetime index to a column
    data.columns = ['Date', 'Close'] # Rename the columns

    data['Close'] = pd.to_numeric(data['Close'], errors='coerce')

    split_ratio = 0.8 # 80% train, 20% test
    split_index = int(len(data) * split_ratio)
    train_data = data.iloc[:split_index].copy()
    test_data = data.iloc[split_index:].copy()

    study = optuna.create_study(direction="maximize",
pruner=optuna.pruners.MedianPruner())
    study.optimize(lambda x: objective(x,train_data), n_trials=trial)

    f = io.StringIO()
    with redirect_stdout(f):
        print('\n'+ticker)
        print(period)
        print(interval)

```

```

        print(study.best_params)

    a = backtest_strategy(
        test_data.copy(),
        study.best_params['short_ma'],
        study.best_params['long_ma'],
        study.best_params['rsi_period'],
        study.best_params['rsi_overbought'],
        period
    )

    output = f.getvalue()

    return output

def greet(ticker_input):

    tickers = [tick.strip().upper() for tick in
ticker_input.split(',')]
    reports = []

    for ticker in tickers:
        globals()['ticker'] = ticker

        report1 = operation(period='3mo', trial=75)
        report2 = operation(period='6mo', trial=125)
        report3 = operation(period='1y', trial=200)

        report = report1 + '\n' + report2 + '\n' + report3

        reports.append(report)

    return '\n\n'.join(reports)

import gradio as gr

# Gradio UI
with gr.Blocks() as demo:
    gr.Markdown("## Ticker Report Generator")
    ticker_input = gr.Textbox(label="Enter Tickers (comma-separated)")
    ticker_output = gr.TextArea(label="Report", lines=10,
max_lines=30)
    ticker_input.submit(greet, inputs=ticker_input,
outputs=ticker_output)

demo.launch()

```

It looks like you are running Gradio on a hosted Jupyter notebook, which requires `share=True`. Automatically setting `share=True` (you

can turn this off by setting `share=False` in `launch()` explicitly).

Colab notebook detected. To show errors in colab notebook, set
debug=True in launch()

* Running on public URL: <https://27f246559a5a82e8be.gradio.live>

This share link expires in 1 week. For free permanent hosting and GPU
upgrades, run `gradio deploy` from the terminal in the working
directory to deploy to Hugging Face Spaces
(<https://huggingface.co/spaces>)

<IPython.core.display.HTML object>

Output Example

USEG

3mo

1h

{'short_ma': 7, 'long_ma': 111, 'rsi_period': 10, 'rsi_overbought': 60}

Open position:

{'entry_date': '2026-04-27 16:00', 'exit_date': None, 'entry_price': 1.01, 'exit_price': None, 'pnl': None}

Number of Trades: 9

Trade List:

{'entry_date': '2026-04-21 09:30', 'exit_date': '2026-04-21 10:30', 'entry_price': 0.7806, 'exit_price': 0.7895, 'pnl': 0.01}

{'entry_date': '2026-04-22 07:00', 'exit_date': '2026-04-22 10:30', 'entry_price': 0.8177, 'exit_price': 0.8177, 'pnl': 0.0}

{'entry_date': '2026-04-22 13:30', 'exit_date': '2026-04-22 15:30', 'entry_price': 0.8201, 'exit_price': 0.8378, 'pnl': 0.02}

{'entry_date': '2026-04-22 18:00', 'exit_date': '2026-04-23 07:00', 'entry_price': 0.8257, 'exit_price': 0.8386, 'pnl': 0.01}

{'entry_date': '2026-04-23 09:30', 'exit_date': '2026-04-23 10:30', 'entry_price': 0.8625, 'exit_price': 0.8608, 'pnl': -0.0}

{'entry_date': '2026-04-24 05:00', 'exit_date': '2026-04-24 15:30', 'entry_price': 0.8881, 'exit_price': 0.927, 'pnl': 0.04}

{'entry_date': '2026-04-27 05:00', 'exit_date': '2026-04-27 08:00', 'entry_price': 0.8999, 'exit_price': 1.4983, 'pnl': 0.6}

{'entry_date': '2026-04-27 12:30', 'exit_date': '2026-04-27 13:30', 'entry_price': 1.16, 'exit_price': 1.135, 'pnl': -0.02}

{'entry_date': '2026-04-27 16:00', 'exit_date': None, 'entry_price': 1.01, 'exit_price': None, 'pnl': None}

Minimum Equity Ever Needed: 0.85 unit of currency

Total P&L on min equity: 0.66 unit of currency

Total End Change on any equity: 1.78 percent in the last 18 days.

USEG

6mo

1h

{'short_ma': 34, 'long_ma': 165, 'rsi_period': 11, 'rsi_overbought': 61}

Open position:

{'entry_date': '2026-04-27 16:00', 'exit_date': None, 'entry_price': 1.01, 'exit_price': None, 'pnl': None}

Number of Trades: 9

Trade List:

{'entry_date': '2026-04-22 06:00', 'exit_date': '2026-04-22 12:30', 'entry_price': 0.8171, 'exit_price': 0.8166, 'pnl': -0.0}

```
{'entry_date': '2026-04-22 13:30', 'exit_date': '2026-04-22 16:00', 'entry_price': 0.8201, 'exit_price': 0.8297, 'pnl': 0.01}
{'entry_date': '2026-04-22 18:00', 'exit_date': '2026-04-23 07:00', 'entry_price': 0.8257, 'exit_price': 0.8386, 'pnl': 0.01}
{'entry_date': '2026-04-23 19:00', 'exit_date': '2026-04-24 04:00', 'entry_price': 0.9042, 'exit_price': 0.8881, 'pnl': -0.02}
{'entry_date': '2026-04-24 05:00', 'exit_date': '2026-04-24 15:30', 'entry_price': 0.8881, 'exit_price': 0.927, 'pnl': 0.04}
{'entry_date': '2026-04-24 18:00', 'exit_date': '2026-04-24 19:00', 'entry_price': 0.9232, 'exit_price': 0.913, 'pnl': -0.01}
{'entry_date': '2026-04-27 04:00', 'exit_date': '2026-04-27 08:00', 'entry_price': 0.89, 'exit_price': 1.4983, 'pnl': 0.61}
{'entry_date': '2026-04-27 11:30', 'exit_date': '2026-04-27 15:30', 'entry_price': 1.11, 'exit_price': 1.07, 'pnl': -0.04}
{'entry_date': '2026-04-27 16:00', 'exit_date': None, 'entry_price': 1.01, 'exit_price': None, 'pnl': None}
```

Minimum Equity Ever Needed: 0.88 unit of currency

Total P&L on min equity: 0.60 unit of currency

Total End Change on any equity: 1.68 percent in the last 36 days.

USEG

1y

1h

```
{'short_ma': 47, 'long_ma': 60, 'rsi_period': 19, 'rsi_overbought': 63}
```

Open position:

```
{'entry_date': '2026-04-27 11:30', 'exit_date': None, 'entry_price': 1.11, 'exit_price': None, 'pnl': None}
```

Number of Trades: 18

Trade List:

```
{'entry_date': '2026-03-03 06:00', 'exit_date': '2026-03-03 13:30', 'entry_price': 1.33, 'exit_price': 1.3811, 'pnl': 0.05}
{'entry_date': '2026-03-03 17:00', 'exit_date': '2026-03-05 09:00', 'entry_price': 1.39, 'exit_price': 1.11, 'pnl': -0.28}
{'entry_date': '2026-03-06 05:00', 'exit_date': '2026-03-06 06:00', 'entry_price': 1.33, 'exit_price': 1.27, 'pnl': -0.06}
{'entry_date': '2026-03-09 14:30', 'exit_date': '2026-03-10 19:00', 'entry_price': 1.095, 'exit_price': 1.0898, 'pnl': -0.01}
{'entry_date': '2026-03-13 07:00', 'exit_date': '2026-03-13 17:00', 'entry_price': 1.06, 'exit_price': 1.06, 'pnl': 0.0}
{'entry_date': '2026-03-16 11:30', 'exit_date': '2026-03-17 07:00', 'entry_price': 1.0379, 'exit_price': 1.04, 'pnl': 0.0}
{'entry_date': '2026-03-26 15:30', 'exit_date': '2026-03-27 09:00', 'entry_price': 1.01, 'exit_price': 1.06, 'pnl': 0.05}
{'entry_date': '2026-03-27 10:30', 'exit_date': '2026-03-31 08:00', 'entry_price': 1.005, 'exit_price': 0.95, 'pnl': -0.05}
```

```
{'entry_date': '2026-04-07 10:30', 'exit_date': '2026-04-07 18:00', 'entry_price': 0.8649, 'exit_price': 0.8399, 'pnl': -0.03}
{'entry_date': '2026-04-15 09:00', 'exit_date': '2026-04-15 15:30', 'entry_price': 0.7151, 'exit_price': 0.7576, 'pnl': 0.04}
{'entry_date': '2026-04-15 19:00', 'exit_date': '2026-04-16 08:00', 'entry_price': 0.7429, 'exit_price': 0.7528, 'pnl': 0.01}
{'entry_date': '2026-04-16 09:30', 'exit_date': '2026-04-16 11:30', 'entry_price': 0.7536, 'exit_price': 0.7445, 'pnl': -0.01}
{'entry_date': '2026-04-16 12:30', 'exit_date': '2026-04-16 13:30', 'entry_price': 0.7573, 'exit_price': 0.7548, 'pnl': -0.0}
{'entry_date': '2026-04-17 06:00', 'exit_date': '2026-04-20 14:30', 'entry_price': 0.75, 'exit_price': 0.7254, 'pnl': -0.02}
{'entry_date': '2026-04-22 12:30', 'exit_date': '2026-04-23 10:30', 'entry_price': 0.8166, 'exit_price': 0.8608, 'pnl': 0.04}
{'entry_date': '2026-04-23 19:00', 'exit_date': '2026-04-24 04:00', 'entry_price': 0.9042, 'exit_price': 0.8881, 'pnl': -0.02}
{'entry_date': '2026-04-24 08:00', 'exit_date': '2026-04-27 08:00', 'entry_price': 0.872, 'exit_price': 1.4983, 'pnl': 0.63}
{'entry_date': '2026-04-27 11:30', 'exit_date': None, 'entry_price': 1.11, 'exit_price': None, 'pnl': None}
```

Minimum Equity Ever Needed: 1.56 unit of currency

Total P&L on min equity: 0.34 unit of currency

Total End Change on any equity: 1.22 percent in the last 73 days.